ECDNN Lab 1: General Matrix Multiplication

[Your Name] Student ID: [Your Student ID]

July 2026

1 Experimental Environment

- Hardware: Apple A18 Pro (arm64)
- Compiler: Apple Clang 21.0.0
- Build flags: `-std=c++17 -O3 -Wall`
- Timing protocol: arithmetic time only, averaged over 32 runs. Matrix initialization and correctness checks are excluded from timed regions.

All matrix programs use `uint32_t`. Its defined wraparound arithmetic avoids signed-overflow undefined behavior, so different loop orders remain mathematically comparable. Every measured implementation is verified against an independently computed reference matrix.

2 Q1: Matrix Multiplication and Memory Access Order

Let A have shape $I \times K$, B have shape $K \times J$, and $C = AB$ have shape $I \times J$. The current Lab 1 requirement, as clarified in the course discussion, varies I, J, and K independently over $\{256, 512, 1024\}$. Thus there are $3^3 = 27$ shapes and four algorithms, for 108 measurements in total.

The format handout presents the 256/512 cases as a compact table. Table 1 follows that layout. The complete 108-row dataset, including every non-square case involving 1024, is supplied as `Q1_results.csv` in the submission folder. Table 2 additionally shows the largest square case.

Table 1: Q1 timings for the 256/512 combinations (seconds).

Algorithm	I	J	K	Average time (s)
`matmul_ijk`	256	256	256	0.039320
`matmul_ikj`	256	256	256	0.016263
`matmul_AT`	256	256	256	0.047428
`matmul_BT`	256	256	256	0.014347
`matmul_ijk`	256	512	256	0.071849
`matmul_ikj`	256	512	256	0.026771
`matmul_AT`	256	512	256	0.091209
`matmul_BT`	256	512	256	0.028565
`matmul_ijk`	256	256	512	0.071600

Algorithm	I	J	K	Average time (s)
`matmul_ikj`	256	256	512	0.029960
`matmul_AT`	256	256	512	0.092601
`matmul_BT`	256	256	512	0.027431
`matmul_ijk`	256	512	512	0.156152
`matmul_ikj`	256	512	512	0.054825
`matmul_AT`	256	512	512	0.240522
`matmul_BT`	256	512	512	0.072494
`matmul_ijk`	512	256	256	0.076837
`matmul_ikj`	512	256	256	0.031326
`matmul_AT`	512	256	256	0.089856
`matmul_BT`	512	256	256	0.030448
`matmul_ijk`	512	512	256	0.149916
`matmul_ikj`	512	512	256	0.058289
`matmul_AT`	512	512	256	0.185444
`matmul_BT`	512	512	256	0.060163
`matmul_ijk`	512	256	512	0.147197
`matmul_ikj`	512	256	512	0.057302
`matmul_AT`	512	256	512	0.189066
`matmul_BT`	512	256	512	0.056760
`matmul_ijk`	512	512	512	0.303563
`matmul_ikj`	512	512	512	0.114815
`matmul_AT`	512	512	512	0.373731
`matmul_BT`	512	512	512	0.115826

Table 2: Largest square Q1 case: $I = J = K = 1024$.

Algorithm	Average time (s)
`matmul_ijk`	2.614750
`matmul_ikj`	0.981076
`matmul_AT`	3.090940
`matmul_BT`	0.891964

Discussion. C/C++ uses row-major storage. In `matmul_ijk`, the inner k loop reads $B[k][j]$ down a column, so consecutive accesses are separated by a row stride. In `matmul_ikj`, the inner j loop walks across contiguous rows of both B and C, while a single value $A[i][k]$ is reused. Therefore `matmul_ikj` has substantially better spatial locality. Transposing B also makes the dot-product access contiguous; transposing A does not address the problematic access to B. The measured timings consistently show that `matmul_ikj` and `matmul_BT` outperform `matmul_ijk` and `matmul_AT`.

3 Q2: im2col Convolution

The required convolution has input shape $1 \times 3 \times 56 \times 56$, 64 filters of shape $3 \times 3 \times 3$, stride 1, and padding 0. Its output shape is $64 \times 54 \times 54$.

The im2col transformation creates one row per output position and one column per flattened receptive-field value:

im2col matrix A	2916×27
Weight matrix B	27×64
GEMM result $C = AB$	2916×64

Here $2916 = 54 \times 54$ and $27 = 3 \times 3 \times 3$. The implementation packs the filters into B, computes the GEMM, then maps $C[\textit{position}][\textit{output channel}]$ back to $\textit{output}[\textit{output channel}][\textit{row}][\textit{column}]$. A direct seven-loop convolution provides the reference output.

Table 3: Q2 timing results.

Measurement	Average time (s)
Naive convolution	0.000477
im2col transform	0.000024
GEMM	0.000719
im2col + GEMM	0.000744

The maximum difference between direct convolution and im2col+GEMM is 0.000000. For this small CPU workload, the handwritten GEMM and the additional im2col copy are slower than the compiler-optimized direct convolution. In production frameworks, im2col is useful because it exposes a large GEMM that can be dispatched to highly optimized BLAS or GPU kernels.

4 Q3: Cache Optimization of GEMM

Q3 uses `matmul_ikj` as the baseline because Q1 established that it already has row-contiguous accesses. Three portable optimizations were implemented:

1. **Tiling** (`matmul_tiled`): multiplies 64×64 blocks so B and C blocks can be reused while they remain in cache.
2. **Tiling plus unrolling** (`matmul_tiled_unroll4`): processes four adjacent output columns per inner-loop iteration.
3. **Tiling plus write-back caching** (`matmul_tiled_writeback4`): keeps four partial C values in scalar accumulators through a K-block, then writes them back once.

Table 4: Q3 timings and speedups. Speedup is $t_{\text{ikj}}/t_{\text{variant}}$.

Algorithm	256 time (s)	Speedup	512 time (s)	Speedup
`matmul_ikj` baseline	0.012034	1.00×	0.086286	1.00×
`matmul_tiled`	0.011239	1.07×	0.089295	0.97×
`matmul_tiled_unroll4`	0.012023	1.00×	0.094827	0.91×
`matmul_tiled_writeback4`	0.002868	4.20×	0.022260	3.88×

Discussion. Tiling gives a small improvement at 256 but is slightly slower at 512 with this block size. This demonstrates that blocking must be tuned for the cache hierarchy and compiler, rather than assumed to be beneficial automatically. Manual unrolling also does not improve this `-O3` build: the compiler already applies aggressive optimizations, while a manually enlarged loop body can add register pressure.

Write-back caching is the strongest optimization. The baseline updates each C element once per k, repeatedly loading and storing it. The write-back kernel loads four C values, accumulates a complete K-block in scalar variables, then writes the four values once. This reduces C-memory traffic and improves temporal locality, giving 4.20× speedup at 256 and 3.88× at 512.

5 Reproducibility and Files

The submission folder contains the report source, the compiled PDF, the two required C++ files, and timing evidence. The experiments can be rerun from the original code directory:

```
./run_q1_benchmark.sh 32 q1_results.csv
./run_im2col_convolution.sh 32
./run_q3_benchmark.sh 32 q3_results.csv
```